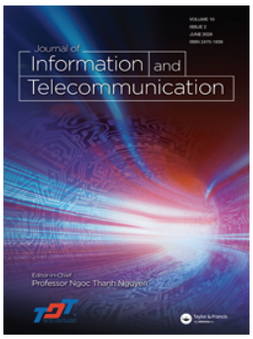


Journal of Information and Telecommunication

ISSN: 2475-1839 (Print) 2475-1847 (Online) Journal homepage: www.tandfonline.com/journals/tjit20

Drift-driven collaborative learning for non-stationary time series: a COVID-19 case study

Khaled Jouini, Farah Jemili & Ouajdi Korbaa

To cite this article: Khaled Jouini, Farah Jemili & Ouajdi Korbaa (25 May 2026): Drift-driven collaborative learning for non-stationary time series: a COVID-19 case study, Journal of Information and Telecommunication, DOI: 10.1080/24751839.2026.2674344

To link to this article: https://doi.org/10.1080/24751839.2026.2674344

Published online: 25 May 2026.

JOURNAL OF INFORMATION AND TELECOMMUNICATION
https://doi.org/10.1080/24751839.2026.2674344

Drift-driven collaborative learning for non-stationary time series: a COVID-19 case study

Khaled Jouini, Farah Jemili and Ouajdi Korbaa

ISITCom, INSIGHT Lab, University of Sousse, Sousse, Tunisia

ABSTRACT

Accurate forecasting of non-stationary time series, such as pandemic data, is particularly challenging due to the presence of *concept drifts*, i.e. changes in the underlying data-generating process over time. Conventional batch learning models often suffer from performance degradation in these settings, as they are trained on past data distributions that may no longer reflect current conditions. While incremental learning enables continuous model updates to better track such changes, it is often regarded as an approximation of fully retrained batch models. In this work, we introduce EFRT-DD (Extremely Fast Regression Tree with Drift Detection) and CDR (Collaborative Drift-Driven Regression), two contributions for adaptive stream regression. EFRT-DD is an eager incremental regression tree that continuously revisits its internal structure to remain aligned with evolving data, while CDR is a collaborative framework that integrates incremental and batch learners to reconcile the drift-adaptation capabilities of incremental models with the predictive strength of batch learners. Using the COVID-19 pandemic as a representative and highly non-stationary real-world benchmark, and under a strict prequential evaluation protocol, our experiments show that EFRT-DD improves upon state-of-the-art incremental regression trees, while the CDR framework further enhances predictive performance compared to standalone incremental or batch models.

ARTICLE HISTORY
Received 18 April 2023
Accepted 9 May 2026

KEYWORDS
Concept drift; incremental learning; collaborative learning; stream regression; pandemic forecasting

1. Introduction

Pandemics will never cease to emerge and threaten both public health and the global economy. Beyond the development of treatments and vaccines, improving epidemiological surveillance and forecasting tools is essential for better preparedness and response. From a machine learning perspective, pandemic forecasting is a challenging example of non-stationary time-series prediction, where the underlying data-generating process evolves over time. This non-stationarity is driven by various real-world factors, including abrupt and recurrent changes in transmission dynamics, shifts in population behaviour, virus mutations, and variations in testing and reporting practices (Moghimi et al., 2023).

CONTACT Khaled Jouini khaled.jouini@isitc.u-sousse.tn, j.khaled@gmail.com ISITCom, INSIGHT Lab, University of Sousse, Route Principale N°1, Hammam Sousse 4011, Tunisia

In machine learning, such temporal changes in the relationship between input data and the learning target are referred to as *Concept Drift* (Bifet, 2010).

The predictive performance of conventional machine learning algorithms, referred to as *batch learners* in the sequel, typically deteriorates under concept drift, as these models remain calibrated on historical data that no longer reflects current patterns (Montiel, Bifet et al., 2018). A common intervention to handle drift is to periodically *retrain the model* to take into account recent data. Besides the computational burden, model retraining raises two significant challenges: (i) identifying the precise moment when a model becomes invalid (the *stability-plasticity dilemma*); and (ii) determining the appropriate volume of new data to collect, as waiting for larger datasets to ensure accuracy inevitably delays the replacement of an obsolete model.

Incremental learning (*a.k.a. online learning* or *lifelong learning*) is an alternative to batch learning, where a model is trained on small amounts of data at a time, rather than all at once (Bifet et al., 2018). In this paradigm, the training phase never ends, and the model is incrementally updated as new data becomes available. By refining parameters using the most recent samples, incremental models maintain higher responsiveness to concept drifts than static batch models. Another key advantage of incremental learning is the '*anytime property*,' which enables a model to provide predictions at any point during its lifecycle (Bifet et al., 2018). This capability is essential when timely forecasts are required and predictions must be made before all the data becomes available, as is often the case in pandemic monitoring scenarios.

Despite these advantages, incremental learning involves inherent trade-offs. Unlike batch models, which leverage a holistic view of the training data to perform global optimization, incremental learners must induce their structure based on a sequential and limited stream of observations. This localized perspective often leads to greedy structural decisions, such as suboptimal splits in decision trees, which may result in lower predictive accuracy compared to batch methods (Bifet, 2010). Existing approaches for learning under concept drift typically treat batch and incremental learning as mutually exclusive paradigms and fail to effectively combine their complementary strengths in a unified, drift-aware manner (Gomes, Montiel et al., 2020; Montiel, Bifet et al., 2018).

To address these limitations, this work proposes a unified approach for learning under concept drift in non-stationary time series. It introduces two main contributions: EFRT-DD (*Extremely Fast Regression Tree with Drift Detection*) and CDR (*Collaborative Drift-Driven Regression*). Specifically, EFRT-DD is an eager incremental regression tree that departs from existing incremental trees by performing early split decisions and continuously re-evaluating them to enhance responsiveness and structural adaptability. Complementing EFRT-DD, CDR is a drift-driven collaborative framework in which incremental and batch regressors are coordinated to operate at different drift adaptation scales. By dynamically selecting between both models based on recent predictive performance, CDR aims to reconcile the responsiveness of incremental learning with the superior predictive strength of batch learners. To validate these contributions, we leverage a multi-country COVID-19 dataset as a representative dataset characterized by heterogeneous and overlapping concept drifts.

The remainder of this paper is organized as follows. Section 2 briefly reviews the main concepts related to incremental learning. Section 3 describes our adaptive, incremental regression tree and our Collaborative Drift-Driven approach. Section 4 provides an

overview of related work. Section 5 outlines the experimental evaluation and examines the main findings. Conclusions and future directions are discussed in Section 6.

2. Preliminaries and key concepts

Machine learning models can be categorized as *white-box* and *black-box*. White-box models provide an explicit representation of how the model arrived at its predictions and are easily interpretable and understandable by humans. Black-box models, on the other hand, tend to be more accurate than white-box models, but are also opaque and difficult to interpret. In sensitive areas (e.g. pandemics forecasting), where explainability is as essential as having accurate predictions, white-box models are often preferred to black-box models (Salah et al., 2023). In the sequel, we are mostly interested in Decision Trees which are among the most popular white-box models (Abid et al., 2022).

2.1. Incremental decision trees

A decision tree (DT) is learned top-down by recursively replacing leaves by test nodes. The recursion is completed when a node is deemed homogeneous enough, or when splitting no longer improves predictions. The crucial decision needed to construct a DT is when to split a node and according to which attribute. The attribute to test at a node is chosen by comparing all available attributes and retaining the one leading to the best homogeneity. Gini index and Information Gain are commonly used in classification trees to measure the level of homogeneity in a node. Conventional (i.e. batch) DTs assume that all training data is available prior to tree induction and scan the entire dataset to discover the best splitting attribute. The aforementioned induction method cannot be adopted directly in stream settings and in environments where only a small fraction of data is accessible during learning (which is typically the case in pandemic forecasts).

The Hoeffding Tree (HT) (Domingos & Hulten, 2000) is the de facto standard in stream mining (Manapragada, Webb et al., 2018) and has inspired many state-of-the-art ensemble and adaptive incremental algorithms (Manapragada, Gomes et al., 2022). The main idea behind HT is that a small fraction of data can often be enough to choose an optimal splitting attribute. This idea is supported by the *Hoeffding Bound* which states that, with probability $1 - \delta$, the true mean of a random variable of range R will not differ from the estimated mean after n independent observations by more than (Domingos & Hulten, 2000):

$$\epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}} \tag{1}$$

For the purpose of deciding which attribute to split on, the random variable being estimated is the difference in information gain between the best and second-best attributes, *resp.* referred to as X_a and X_b in the sequel. As shown in Algorithm 1, if the computed difference of information gains between X_a and X_b is higher than ϵ: $G(X_a) - G(X_b) > \epsilon$, the algorithm asserts with confidence $1 - \delta$, that X_a will always remain a better split option than X_b.

The *Hoeffding Adaptive Tree* (Bifet & Gavaldà, 2007) and the FIMT-DD (Fast Incremental Model Tree with Drift Detection (FIMT-DD) (Ikonomovska et al., 2011) (discussed in Subsection 2.2), are popular extensions of HT that incorporate special mechanisms for handling concept drifts.

Algorithm 1: The Hoeffding Tree algorithm (Domingos & Hulten, 2000)

Input: S, a sequence of samples
δ, one minus the desired probability of choosing the correct attribute at a given node
G(.), a split evaluation function
Output: HT, a decision tree
1 Let HT be a tree with a single leaf (root)
2 **for all** $(\vec{x}, y) \in S$ **do**
3 Sort $(\vec{x}, y)$ to leaf l using HT
4 **if** l is not pure (samples seen so far at l are not all of the same class)
5 Compute $G_l(X_i)$ for each attribute $X_l \setminus \{X_\emptyset\}$
6 Let X_a be the attribute with highest G_l
7 Let X_b be the attribute with second-highest G_l
8 Compute ε using Equation (1)
9 **if** $G_l(X_a) - G_l(X_b) > \epsilon$ and $X_a \neq X_\emptyset$ **then**
10 Split l on X_a
11 **for all** each branch
12 Initialize new leaf 13 end forall
14 end if
15 end if
16 end forall

2.2. Incremental regression trees

The goal of a regression task is to learn a model M that predicts a real value, and not one of a discrete set of values as in classification (Tran-Nguyen et al., 2020). Formally, let S be a continuous stream of data: $S = \{(\vec{x}^t, y^t)\}$, where $\vec{x}^t$ is a feature vector, $y^t \in \mathbb{R}$ is the target variable and t the arrival timestamp. The goal is to incrementally learn $M : \vec{x} \rightarrow y$ as new data becomes available (Gomes, Barddal et al., 2018). The predicted value of M is denoted as $\hat{y}$. When the actual value y gets revealed, the performance P is measured according to a loss function L: $P(M) = L(y, \hat{y})$.

The Fast Incremental Model Tree with Drift Detection (FIMT-DD) (Gomes, Barddal et al., 2018; Ikonomovska et al., 2011, 2014), is one of the most efficient incremental regression trees (Gomes, Montiel et al., 2020). FIMT-DD behaves similarly to HT and works by incrementally updating a tree structure as new data arrives, ranking features based on their variance *w.r.t* the target variable, and making splits if the two best-ranked features differ by at least the Hoeffding Bound. The salient features of FIMT-DD are synthesized in the following points.

Splitting Criterion FIMT-DD uses the Standard Deviation Reduction (SDR) measure as splitting criterion, i.e. the attribute to split on is the one allowing the largest reduction in variance. Given a leaf l where a sample of size N has been observed, a binary split over an attribute X divides the samples in l in two disjoint subsets l_l and l_r, with sizes N_l and N_r. The Standard Deviation Reduction $SDR(X)$ is then computed as follows (Ikonomovska et al., 2011).

$$\mathrm{SDR}(X) = sd(l) - \left(\frac{N_l}{N} sd(l_l) + \frac{N_r}{N} sd(l_r)\right) \tag{2}$$

$$sd(node) = \sqrt{\frac{1}{N}\left(\sum_{i=1}^{N}(y_i - \bar{y})^2\right)} = \sqrt{\frac{1}{N}\left(\sum_{i=1}^{N} y_i^2 - \frac{1}{N}\left(\sum_{i=1}^{N} y_i\right)^2\right)}, \tag{3}$$

where *sd*(*node*) is the Standard Deviation of the target variable in *node*. Let X_a and X_b be respectively the best split attribute and the second-best split attribute. Similarly to HT, FIMT-DD uses the Hoeffding Bound to control the risk that, as data arrives, the merit of splitting on X_b exceeds the merit of splitting on X_a. In practice, before splitting a node, FIMT-DD waits until the following condition is met.

$$\frac{\mathrm{SDR}(X_b)}{\mathrm{SDR}(X_a)} < 1 - \epsilon \tag{4}$$

Linear model at the leaves FIMT-DD trains a perceptron at each leaf of the tree. The weights of these perceptrons are continuously updated as new data arrives using the incremental stochastic gradient descent method and with the objective of minimizing the mean squared error. Besides their proven effectiveness, perceptrons have the crucial advantage of naturally adapting to drifts (Ikonomovska et al., 2011).

Drift handling FIMT-DD uses the Page-Hinkley (PH) change detection test (Mouss et al., 2004) at inner nodes to detect changes in the error rate. When a change is detected in an inner node *inner*, an alternate tree rooted at *inner* is grown with new incoming instances: every new instance that reaches *inner* is used for growing both subtrees. The new subtree replaces the original subtree when (and if) it performs better.

3. Collaborative drift-driven regression (CDR)

3.1. Drift-Driven models management

Inspired by the work of Montiel, Bifet et al. (2018) on fast and slow classifiers, the main idea behind our drift-driven collaborative approach is to jointly use incremental learning and batch learning to leverage the advantages of both: (i) the accuracy of batch regressors; and (ii) the inherent adaptability to changes of incremental regressors and their anytime property. As depicted in Figure 1, we consider training as a continuous process in which incremental and batch regression coexist. Incremental learning is used to continuously and incrementally train and refine a regressor *I*, as new samples

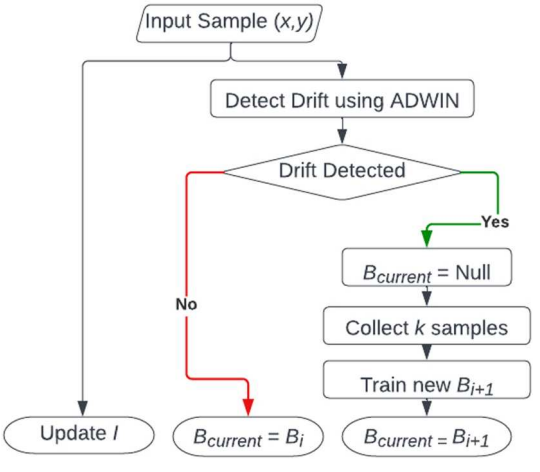


Figure 1. CDR – Learning process.

become available. Batch learning is used to train a sequence of (batch) regressors $\{B_1, B_2, \ldots, B_n\}$. As illustrated in Figure 1, whenever a drift is detected, the current batch regressor B_i is invalidated and is subsequently replaced by a new regressor B_{i+1}. While I is trained on single samples as they arrive, a batch regressor B_i is trained on a (micro-) batch M_i containing the k most recent samples. This implies that the training process of B_{i+1} is deferred until k samples are gathered (Figure 1).

Our Drift-Driven Collaborative Regression approach uses *ADaptive WINdowing* (ADWIN) (Bifet, 2010; Bifet & Gavaldà, 2007) for drift detection. The basic idea behind ADWIN is to maintain a variable-length sliding window W which increases in size as long as no drift is detected. To detect a drift, ADWIN repeatedly partitions W into two adjacent sub-windows W_0 and W_1 and compares their average to decide whether they are likely to originate from the same distribution. If W_0 and W_1 exhibit distinct enough averages and are of sufficient size, then a drift is detected and W is shrunk by dropping W_0 items from the window. In practice, ADWIN tests if the difference between the averages of W_0 and W_1 is larger than a variable value ϵ_{cut} computed as Bifet and Gavaldà (2007):

$$m = \frac{2}{\frac{1}{|W_0|} + \frac{1}{|W_1|}} \tag{5}$$

$$\epsilon_{cut} = \sqrt{\frac{1}{2m} \ln \frac{4|W|}{\delta}}, \tag{6}$$

where m is the harmonic mean of $|W_0|$ and $|W_1|$. Unlike other existing drift detectors, ADWIN is assumption-free. Its only parameter is a confidence bound $\delta \in [0, 1]$, which enables adjusting the sensitivity to drifts.

The inference process in CDR is depicted in the flowchart of Figure 2. As illustrated in Figure 2, when a batch regressor B_i is invalidated and until k samples are collected to train a new batch regressor B_{i+1}, only the incremental regressor I is used for inference. When a new batch regressor $B_{current}$ becomes available, CDR tracks the predictive performance of I and $B_{current}$ over a sliding window W containing the most recent observations. The top-

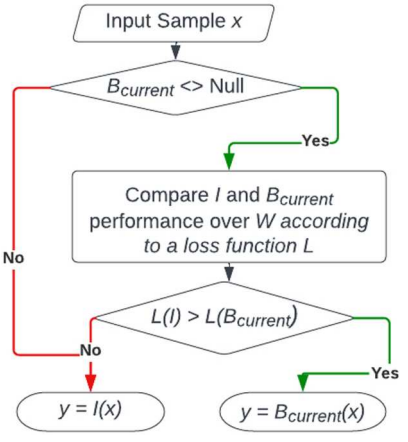


Figure 2. CDR – Inference process.

performing model over W is then selected for inference. The aforementioned process is repeated for each new incoming instance.

3.2. Extremely fast regression tree with drift detection (EFRT-DD)

Although several incremental and online algorithms have been proposed, most of them focus on classification tasks and ignore regression tasks (Gomes, Montiel et al., 2020). In this work, we propose a new incremental regression tree, called the *Extremely Fast Regression Tree with Drift Detection*. The proposed tree is an adaptation of the Extremely Fast Tree presented in Manapragada, Webb et al. (2018) and Manapragada, Gomes et al. (2022) to the regression task and to drift detection. The EFRT-DD can also be seen as an improvement of the FIMT-DD (Ikonomovska et al., 2011), which is considered as one of the state-of-the-art incremental regression trees (Gomes, Montiel et al., 2020). Like FIMT-DD, EFRT-DD trains a perceptron at each leaf and uses the PH test at inner nodes. Different to FIMT-DD, EFRT-DD splits a node as soon as it is sufficiently confident that the split is useful, and subsequently revisits that decision if, as data arrives, it becomes evident that a better split is available.

In practice, HT and its variants (including the FIMT-DD) delay a split until they are confident enough that the current best split attribute X_a will always remain a better option than the second-best split attribute X_b (regardless of the merit of the split). As pointed out in Manapragada, Webb et al. (2018), this 'lazy' induction strategy has significant drawbacks. First, delaying splits can affect predictive performance because the tree being built is also used for inference. Second, in HT and its variants (including the FIMT-DD), the Hoeffding Bound controls the risk that, as data arrives, X_b becomes a better split attribute than X_a. However, the used test does not control the risk that a third attribute X_c becomes a better split attribute than X_a. In such cases, there is no recourse to alter the tree, as in HT and its variants split decisions are irrevocable. Third, if the information is uniformly distributed among attributes ($SDR(X_a)$ and $SDR(X_b)$ are close in value), the FIMT-DD will struggle to split and might have to delay the split and/or to resort to using a tie-breaking that gives no probabilistic guarantee. Finally, as split decisions are never revisited, the FIMT-DD will increasingly diverge from the asymptotic batch learner as the tree size increases.

Algorithm 2: AttemptToSplit

Input: l, a leaf node
$n_{\min}$, grace period
1 Let N be the number of samples seen in l
2 **if** N mod $n_{\min} = 0$ **then**
3 Compute $SD(l)$
4 Compute $SumSDchilds(X)$ for each attribute X_i
5 Let X_a be the attribute with the highest SD_{ratio}
6 Compute ϵ using Equation (1)
7 **if** $\frac{SumSDchilds(X_a)}{SD(l)} < 1 - \epsilon$ **then**
8 Split l on X_a
9 **for all** each branch **do**
10 Initialize a new leaf 11 **end forall**
12 **end if**
13 **end if**

To overcome the downsides of the FIMT-DD, we adopt an 'eager' induction strategy inspired by the work of Manapragada, Webb et al. (2018) and Manapragada, Gomes et al. (2022). The Hoeffding bound is used in EFRT-DD to determine, with the required level of confidence, whether the merit of splitting on the current best attribute exceeds the merit of not having a split, or the merit of the current split attribute. The splitting and reevaluation strategies of the EFRT-DD are presented in Algorithms 2 and 3, respectively. As shown in these algorithms, the EFRT-DD uses the *Standard Deviation Ratio* as its splitting criterion. The SD_{ratio} is defined as follows (Bifet et al., 2018).

$$SD_{ratio}(X) = \frac{sd(parent) - sumSDchilds(X)}{sd(parent)} \tag{7}$$

$$sumSDchilds(X) = \frac{N_l}{N} sd(l) + \frac{N_r}{N} sd(r) \tag{8}$$

where, X denotes the attribute being evaluated, $sd(parent)$ is the standard deviation of the target variable in the node *parent* (the node that we attempt to split), N is the number of samples in *parent*, $sd(child_i)$ is the standard deviation of the target variable in the ith child of *parentl*, $\frac{N_i}{N}$ is the proportion of samples in the parent node that belong to the ith child node and $SumSDchilds(X)$ is the weighted sum of the standard deviations in the child nodes of *parent*.

Algorithm 3: ReevaluateBestSplit

Input *int*, an internal node
$r_{\min}$, reevaluation period
1 Let $X_{current}$ be the current split attribute in *int*
2 Let N be the number of samples seen in *int*
3 **if** N mod $r_{\min} = 0$ **then**
4 Compute SD_{ratio} for each attribute X_i
5 Compute $\epsilon_{\hat{\jmath}}$ using Equation (1)
6 **if** $\frac{SumSDchilds(X_a)}{SD(int)} > 1 - \epsilon_{\hat{\jmath}}$ **then**
7 *KillSubTree*(*int*)
8 Replace *int* with a new leaf node *l*
9 Initialize *l*
10 **else** 11 Compute ϵ_{ratio} using Equation (1)
12 **if** $\frac{SD_{ratio}(X_{current})}{SD_{ratio}(X_a)} < 1 - \epsilon_{ratio}$ **then**
13 **Split** *int* on X_a
14 **for all** each branch
15 Initialize a new leaf
16 **end forall**
17 **end if**
18 **end if**
19 **end if**

As shown in Algorithm 3, when EFRT-DD is sufficiently confident that the current split is suboptimal (either $X_{current} \neq X_a$ or $X_{current} = X_a$, but $X_{current}.splitTest \neq X_a.splitTest$), it performs a new split on X_a to replace the old split. Similarly, if the current split is not significantly better than a non-split, the corresponding subtree is pruned, and the internal node is replaced with a new leaf node.

4. Related work

Modelling and predicting the COVID-19 pandemic has attracted extensive research. Approaches range from classical statistical models, such as ARIMA (Camargo et al., 2022), to advanced deep learning architectures (Mydukuri et al., 2022; Tran et al., 2022). These approaches can be broadly categorized into two families (Miralles-Pechuán et al., 2023): *compartmental* (mechanistic) models and *machine learning* models (*a.k.a.* curve-fitting models). In the following, we discuss representative compartmental and machine learning approaches that have addressed non-stationarity.

Compartmental models, such as SEIRD, typically partition the population into *Susceptible*, *Exposed*, *Infected*, *Recovered*, and *Dead* compartments, with transitions between these compartments governed by differential equations. While these models inherently capture the temporal dynamics of disease spread, they traditionally rely on static parameters (e.g. transmission and recovery rates) that assume a stable environment. To address this limitation, recent research has increasingly integrated machine learning to calibrate or refine these underlying dynamics in a more data-driven manner. For instance, Camargo et al. (2022) proposed a dual-component architecture combining a genetic algorithm with ARIMA models to identify the best subset of predictors for each SEIRD variable. Ensemble learning is then used to select the best-performing regressor, with new models being built whenever predictive accuracy drops. Despite enabling some dynamic model selection, this approach lacks explicit drift-aware mechanisms for model lifecycle management. Within the same mechanistic family, Nguyen et al. (2022) proposed BeCaked, which integrates a SIRD compartmental structure with a Variational-LSTM Autoencoder to provide intrinsic explainability. Although BeCaked incorporates a threshold-based fine-tuning strategy, its adaptation to shifting pandemic phases remains constrained by periodic retraining.

Turning to machine learning approaches, several studies have addressed non-stationarity through adaptive modelling strategies. Miralles-Pechuán et al. (2023) showed that training models on clusters of countries with similar pandemic dynamics (identified using Dynamic Time Warping – DTW) significantly outperformed both single-country and global training strategies. Miralles-Pechuán et al. (2023) also compared batch algorithms, such as LSTM, with incremental models like Hoeffding Trees, finding that batch learners often achieved higher accuracy. However, the study does not incorporate explicit drift detection to manage the model lifecycle or coordinate dynamically between these learning paradigms. In Cramer et al. (2022), a large-scale evaluation of quantile-based ensemble probabilistic forecasts was performed, aggregating predictive distributions from diverse mechanistic and deep learning architectures. This study showed that such ensemble strategies significantly improve forecast reliability. Nevertheless, these ensembles typically rely on static aggregations and lack mechanisms to dynamically coordinate model updates in response to distributional changes. To overcome the limitations of single-paradigm models, recent research has turned to hybrid architectures. In Kumar and Susan (2025), a framework was proposed that integrates high-order fuzzy time series with context-augmented LSTM variants and Particle Swarm Optimization (PSO). This approach demonstrated strong capacity to model successive pandemic waves through optimized hyperparameter tuning. However, the method remains anchored in

a batch-learning paradigm, and thus does not enable seamless, instance-by-instance structural adaptation.

The literature reviewed above highlights a persistent limitation: model adaptation remains primarily a discrete retraining process rather than a continuous, drift-aware mechanism. Furthermore, even within incremental learning, most regression trees rely on lazy induction, delaying structural adaptation. Our work addresses these gaps in two complementary ways. First, EFRT-DD combines eager induction with continuous structural adaptation to enhance predictive responsiveness. Second, the CDR framework implements an automated, drift-driven lifecycle management system that coordinates incremental and batch learners within a unified streaming architecture.

5. Experimental evaluation

5.1. Tools and datasets

We considered the dataset 'Coronavirus Pandemic (COVID-19)' (Capodici et al., 2022), provided by Our World in Data (OWID), one of the leading scientific online organizations publishing global data and research on the COVID-19 Pandemic. The original dataset includes daily information about the pandemic in 219 countries, starting from January 2020. Our target variables, the *daily new confirmed* COVID-19 *cases* and *deaths* per million people, are respectively referred to as *new_cases* and *new_deaths* in the sequel. We model the evolution of *new_cases* and *new_deaths* as function of the previously reported cases. For each country C and each record of C with a timestamp t, we consider the number of cases per million reported at 8 time points: t minus 1 week, t minus 2 weeks,.., t minus 8 weeks. [1] The obtained dataset contains nine input variables, $\approx$ 177k samples and covers the period starting from March 28, 2020 to November 30, 2022 ($\approx$ 31 months). When restricted to Tunisian data, the dataset contains $\approx$ 910 samples.

At the current state of our work, we implemented EFRT-DD and CDR using MOA (*Massive On line Analysis*) (Bifet et al., 2018), Scikit-Multiflow (Montiel, Read et al., 2018) and Scikit-Learn (Pedregosa et al., 2011). We configured CDR using EFRT-DD for incremental learning, the Decision Tree (Pedregosa et al., 2011) for batch learning, and ADWIN for drift detection (Montiel, Read et al., 2018). All methods were run using their default settings, and no special tuning was done. Specifically, ADWIN was used with its default confidence value of $d = 0.2\%$ (Montiel, Read et al., 2018). In our experiments, we compared the performance of batch and incremental models using a sliding window of one week and selected the best performing model to make predictions for the current input. Batch models were trained on windows of three weeks, with two weeks before and one week after a detected drift. The models' performance was evaluated using two metrics: the Mean Absolute Error (MAE) and the Root Mean Squared Error (RMSE). In general, RMSE is more sensitive to large errors and outliers, while MAE is considered to be more interpretable. To ensure a fair 'out-of-the-box' comparison and assess reproducibility, all methods were run using their default settings without special tuning. In particular, ADWIN was used with its default confidence value of $\delta = 0.2\%$. While our framework proved relatively robust in these settings, it is worth noting that, as typical for streaming drift detectors, models remain sensitive to the drift detection threshold δ, which affects sensitivity to small fluctuations.

5.2. Results and discussion

We conducted our experiments with three objectives in mind: (i) Put ourselves in the shoes of a country grappling with a pandemic and needing to anticipate its progression; (ii) Evaluate the effectiveness of our incremental regressor, the EFRT-DD, in comparison with established incremental regression trees; and (iii) Demonstrate that CDR is an effective collaboration strategy between batch and incremental learning for modelling and forecasting the evolution of a pandemic. A large number of experiments have been performed to demonstrate the effectiveness of EFRT-DD and CDR. Due to the lack of space, only few results are presented herein.

5.2.1. CDR vs. Incremental learners

The first set of experiments compares the performance of EFRT-DD and CDR to established incremental methods. Conventionally, incremental methods are tested using a *prequential evaluation* scheme, which involves processing each data point sequentially and testing the model's performance on the most recent data point. This method ensures that the model is tested on new data that it has not yet seen and allows for the model to adapt to changes in the data distribution over time.

The results of the prequential evaluation are summarized in Table 1, and partially illustrated in Figures 3 and 4. Reported model run times are average of three consecutive runs. As shown in Table 1, EFRT-DD and FIMT-DD outperform HAT and HT in almost all cases. When compared to FIMT-DD, EFRT-DD achieves an improvement of 2.38%, 9.78%, 4.14% and 4.18% (*resp.* 122.55%, 168.67%, 13.82% and 27.35%) with regards to $RMSE_{Deaths}$, MAE_{Deaths}, $RMSE_{Cases}$ and MAE_{Cases} on world data (*resp.* single-country data). CDR allows

Table 1. MAE and RMSE achieved by CDR and incremental learners.

	Daily New Confirmed Deaths (per 1M)			
	Model	Time	$RMSE_{Deaths}$	MAE_{Deaths}
World	HT	12m31s	2.01	1.21
	HAT	11m36s	2.03	1.23
	FIMT-DD	**8m27s**	1.72	1.01
	EFRT-DD	44m27s	1.68	0.92
	CDR	–	**1.66**	**0.88**
Tunisia	HT	4.89s	2.47	2.44
	HAT	4.45s	2.46	2.43
	FIMT-DD	**4.28s**	2.27	2.23
	EFRT-DD	22.23s	1.02	0.83
	CDR	–	**0.98**	**0.74**
	Daily New Confirmed Cases (per 1M)			
	Model	Time	$RMSE_{Cases}$	MAE_{Cases}
World	HT	5m28s	174.1	91.51
	HAT	6m19s	150.12	80.67
	FIMT-DD	**6m8s**	137.75	74.53
	EFRT-DD	39m56s	132.27	71.54
	CDR	–	**126.86**	**68.11**
Tunisia	HT	2.14s	61.56	58.71
	HAT	**1.42s**	59.35	56.68
	FIMT-DD	1.56s	56.50	52.29
	EFRT-DD	9.38s	49.64	41.06
	CDR	–	**44.37**	**36.83**

Note: (HT: Hoeffding Tree; HAT: Hoeffding Adaptive Tree; FIMT-DD: Fast Incremental Model Tree with Drift Detection; EFRT-DD: Extremely Fast Regression Tree with Drift Detection; CDR: Collaborative Drift-Driven Regression.)

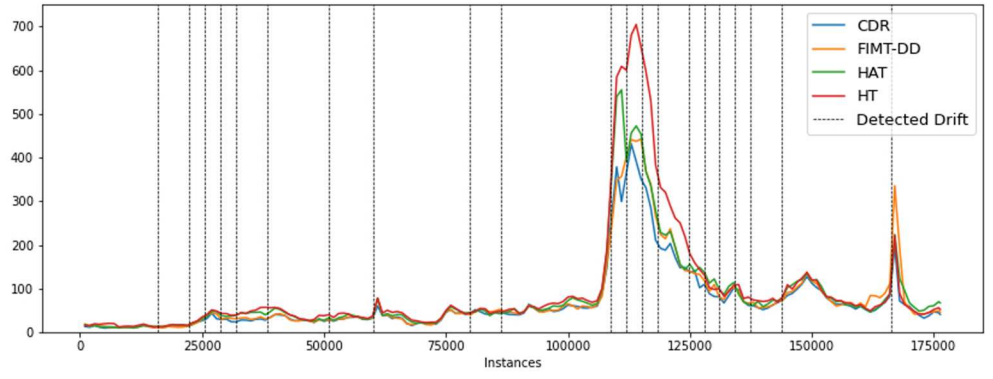


Figure 3. World – Daily New Confirmed Cases – MAE achieved by CDR and incremental learners.
(HT: Hoeffding Tree; HAT: Hoeffding Adaptive Tree; FIMT-DD: Fast Incremental Model Tree with Drift Detection; EFRT-DD: Extremely Fast Regression Tree with Drift Detection; CDR: Collaborative Drift-Driven Regression.)

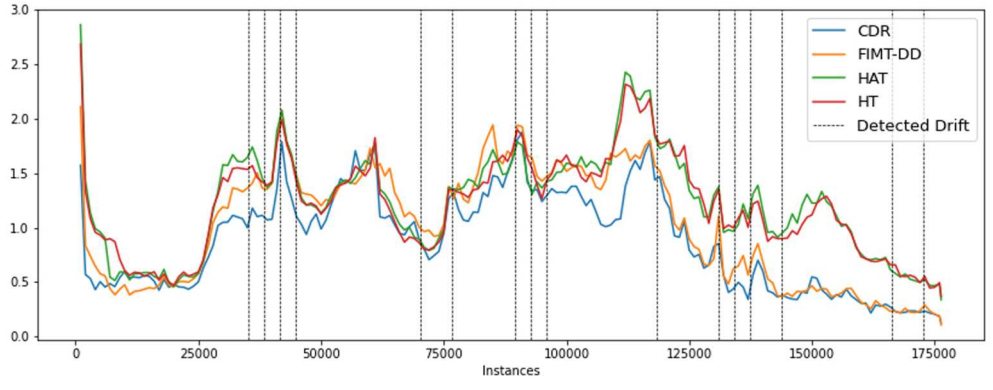


Figure 4. World – Daily New Confirmed Deaths – MAE achieved by CDR and incremental learner.
(HT: Hoeffding Tree; HAT: Hoeffding Adaptive Tree; FIMT-DD: Fast Incremental Model Tree with Drift Detection; EFRT-DD: Extremely Fast Regression Tree with Drift Detection; CDR: Collaborative Drift-Driven Regression.)

further improvements when compared to FIMT-DD: 3.61%, 14.77%, 8.58% and 9.43% (*resp.* 131.63%, 149%, 27.34% and 41.98%) with regards to $RMSE_{Deaths}$, MAE_{Deaths}, $RMSE_{Cases}$ and MAE_{Cases} on world data (*resp.* single-country data).

The counterpart of the good predictive performance of EFRT-DD is its relative slowness. On average, EFRT-DD is 5 to 6 times slower than FIMT-DD. The rationale behind this is that EFRT-DD continuously revisits its split decisions to readjust the model (by discarding outdated splits). Such revisions slow the training process but help in improving the overall predictive performance. The relative slowness of EFRT-DD with regards to FIMT-DD is acceptable in daily epidemiological monitoring scenarios where updates occur at low frequency.

5.2.2. CDR vs. Batch regression tree

Our second set of experiments compares the performance of a conventional batch regression tree against CDR. The prequential evaluation approach, commonly used in

incremental learning, can be applied to batch learning by repeatedly retraining and re-evaluating the model. For each training/testing round, the dataset is split into a training set and a test set in an order-preserving fashion. Instances used for testing the ith batch model are appended to the training set of the $(i+1)$th model. The evaluation of the $(i+1)$th batch model is then performed on instances that arrived after its training and before the training of a new model. The process is repeated for multiple rounds until all the data has been used for both training and testing (except the first batch of data, which is only used for training, and the last batch, which is only used for testing). In Miralles-Pechuán et al. (2023), training/evaluation rounds are referred to as *milestones*. In this study, we followed the aforementioned process (used also in (Miralles-Pechuán et al., 2023)) and adopted a realistic scenario where a new batch model is trained from scratch every ≈ 3 months, resulting in a set of 9 milestones (and, hence, 9 batch models).

Tables 2 and 3 report the performance of the considered batch models and of CDR over the 9 milestones. As illustrated in Tables 2 and 3 CDR by far outperforms the corresponding batch model and respectively achieves an average improvement of 131.61%, 89.54%, 63.23% and 21.85% (*resp.* 165.78%, 167.08%, 137.88% and 93.06%) with regards to *resp.* $RMSE_{Deaths}$, MAE_{Deaths}, $RMSE_{Cases}$ and MAE_{Cases} on world data (*resp.* single-country data). Overall, experimental results confirm that our collaborative drift-driven approach yields better results than those attained by each of the contributing models separately. Experimental results also confirm that in the particular case of predicting the evolution of a pandemic, a drift-driven retraining approach allows better predictive performance than retraining at fixed intervals.

Table 2. Daily new confirmed cases per 1M – MAE and RMSE achieved by CDR and the batch decision tree.

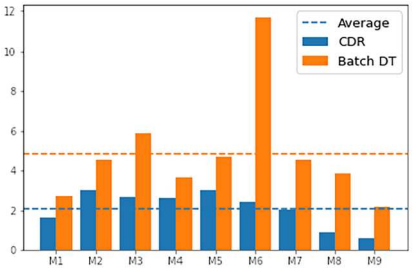
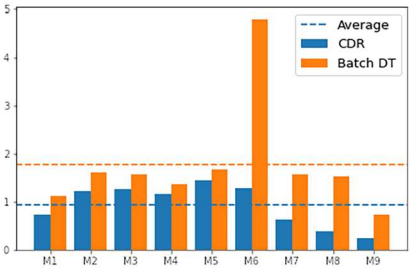
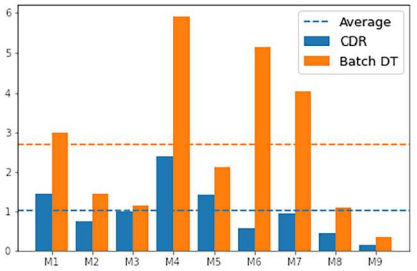
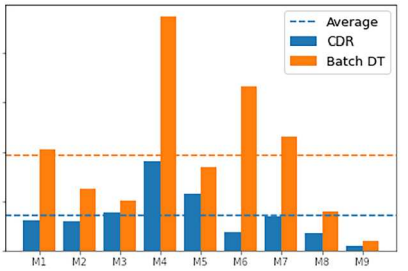
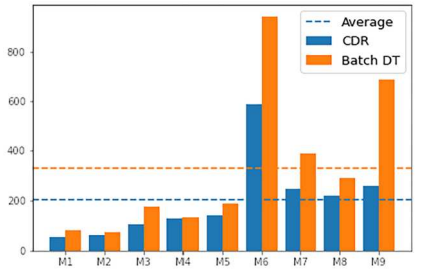
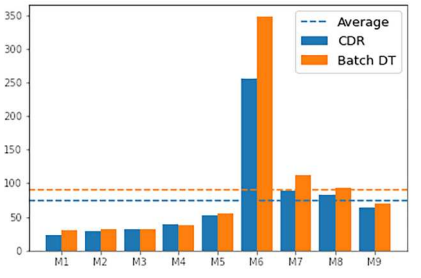
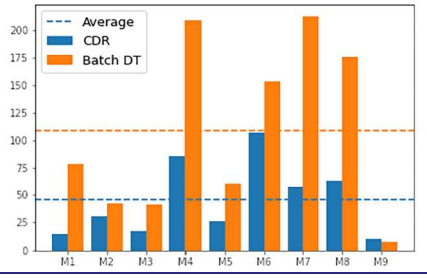
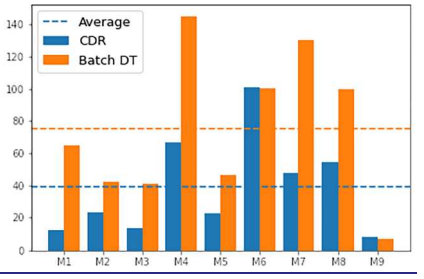
	Daily New Confirmed Cases (per 1M)	
	$RMSE_{Cases}$	MAE_{Cases}
World		
Tunisia		

Table 3. Daily new confirmed deaths per 1M – MAE and RMSE achieved by CDR and the batch decision tree.

	Daily new confirmed deaths (per 1M)	
	$RMSE_{Deaths}$	MAE_{Deaths}
World		
Tunisia		

5.2.3. Discussion

The results presented in Subsections 5.2.1 and 5.2.2 lead to the following methodological and practical conclusions.

(1) *Structural reactivity through eager induction.* Comparative results indicate that conventional lazy incremental trees are limited by delayed splitting and irreversible structural decisions. In such models, early suboptimal choices may persist and progressively degrade predictive performance. EFRT-DD addresses these drawbacks through an eager induction strategy, splitting nodes as soon as a division is statistically preferred over no split. Furthermore, the model continuously reevaluates and revises its internal structure to remain aligned with the underlying data. As suggested by the prequential evaluation, this structural flexibility helps mitigate both short-term underfitting and long-term structural bias.

From a practical perspective, such dual capacity appears especially relevant in scenarios prone to abrupt distributional shifts, such as volatile financial markets or sudden anomalies in network traffic patterns. In these contexts, delayed or irreversible structural updates can lead to sustained performance degradation, whereas the ability to rapidly revise model structure enables EFRT-DD to better track evolving patterns.

(2) *Drift-triggered collaborative adaptation: Recent vs. cumulative retraining.* Systematic retraining on entire historical datasets may be effective when the underlying concept remains relatively stable. In highly non-stationary time series, however, such cumulative retraining can dilute the influence of recent and potentially more

relevant observations. Furthermore, cumulative retraining entails a growing computational burden as data volume expands. CDR addresses these limitations by combining two complementary adaptation scales. The incremental component provides high-frequency responsiveness to absorb abrupt changes. Simultaneously, the drift-triggered batch component acts as a stabilizer, retraining exclusively on recent data windows. This behaviour is further reinforced by dynamically and continuously selecting the best-performing model based on the current regime.

From a practical perspective, this dual-track strategy is particularly relevant for environments where abrupt and gradual changes coexist, such as energy demand forecasting. In these settings, the capacity to reconcile immediate responsiveness (e.g. weather-driven shocks) with structural stability (e.g. long-term seasonal transitions) is essential for ensuring accurate predictions without requiring extensive historical data processing.

(3) *Temporal coherence in multi-source learning*. Training CDR on data from a single country led to higher performance gains than multi-country training. This can be attributed to the temporal heterogeneity of concept drifts across regions. When multiple sources are combined, these distinct drift phases can mask local distributional changes and dilute the drift signal. In contrast, country-specific training preserves temporal coherence, enabling clearer drift detection and more targeted responses. This observation is consistent with the findings reported in Miralles-Pechuán et al. (2023).

More broadly, these results suggest that in multi-source environments characterized by asynchronous dynamics, such as federated monitoring or distributed sensing, localized learning strategies may be preferable to naive aggregation, which can generate conflicting adaptation signals.

Taken together, these results highlight the effectiveness of combining structurally reactive incremental models with drift-aware coordination mechanisms to provide a flexible response to heterogeneous non-stationary dynamics.

6. Conclusion

Accurately forecasting time-evolving phenomena remains a fundamental challenge due to the prevalence of concept drifts. In this work, we addressed this non-stationarity through two complementary contributions: EFRT-DD, which introduces a continuously revisable incremental tree structure, and CDR, a collaborative framework designed to reconcile incremental agility with the predictive strength of batch learners. Experimental evaluation on a highly non-stationary pandemic benchmark shows that EFRT-DD improves upon state-of-the-art incremental trees, while the CDR framework further enhances performance by dynamically balancing stability and plasticity. Despite these promising results, several limitations should be acknowledged. First, the evaluation relies on retrospectively consolidated datasets, which may not fully reflect real-time operational uncertainties such as reporting delays or data revisions. Second, the current focus on pandemic data leaves the external validity of the approach across other non-stationary domains yet to be experimentally confirmed. Lastly, the computational overhead induced by eager splitting may pose a challenge for deployment in high-throughput environments.

Beyond the specific case study considered, this work contributes to a broader understanding of how learning systems can remain effective under persistent distributional change. These findings open several promising research directions. To address the aforementioned constraints, future work will focus on evaluating the framework's resilience under real-world operational delays to refine its practical deployment in live monitoring systems. Second, subgroup-based learning strategies will be investigated to better account for asynchronous drifts across multi-source environments. Finally, subsequent efforts will explore the integration of neural forecasting models in streaming settings and extend the CDR framework with further coordination paradigms, such as meta-learning.

Note

1. We do not consider the cases reported less than a week before *t*. Although including such observations could lead to more accurate models, they are not practical for timely policy responses.

Author contributions

CRediT: **Khaled Jouini:** Conceptualization, Methodology, Software, Validation, Visualization, Writing - review & editing; **Farah Jemili:** Conceptualization, Methodology, Writing - original draft, Writing - review & editing; **Ouajdi Korbaa:** Conceptualization, Methodology, Writing - original draft, Writing - review & editing.

Disclosure statement

No potential conflict of interest was reported by the authors.

ORCID

Khaled Jouini http://orcid.org/0000-0001-5049-4238
Farah Jemili http://orcid.org/0000-0001-7511-1221
Ouajdi Korbaa http://orcid.org/0000-0003-4462-1805

References

Abid, A., Jemili, F., & Korbaa, O. (2022). Distributed architecture of an intrusion detection system in industrial control systems. In *Advances in computational collective intelligence – 14th International Conference, ICCCI 2022, September 28-30, 2022, Proceedings* (Vol. 1653 of *Communications in Computer and Information Science*, pp. 472–484). Springer.

Bifet, A. (2010). Adaptive stream mining: Pattern learning and mining from evolving data streams. *Frontiers in Artificial Intelligence and Applications*, *207*, 1–212.

Bifet, A., & Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining* (Vol. 7, pp. 443–448). Society for Industrial and Applied Mathematics (SIAM).

Bifet, A., Gavaldà, R., Holmes, G., & Pfahringer, B. (2018). *Machine learning for data streams with practical examples in MOA*. MIT Press.

Camargo, E., Aguilar, J., Quintero, Y., Rivas, F., & Ardila, D. (2022). An incremental learning approach to prediction models of seird variables in the context of the COVID-19 pandemic. *Health and Technology*, *12*(4), 2190–7196. https://doi.org/10.1007/s12553-022-00668-5

Capodici, A., Gori, D., & Lenzi, J. (2022). Deaths, countermeasures, and obedience: How countries' non-pharmaceutical measures have quelled the COVID-19 death toll. *Frontiers in Public Health*, *10*, 1–4. https://doi.org/10.3389/fpubh.2022.934309

Cramer, E. Y., Ray, E. L., Lopez, V. K., Bracher, J., Brennen, A., A. J. C. Rivadeneira, Gerding, A., Gneiting, T., House, K. H., Huang, Y., Jayawardena, D., Kanji, A. H., Khandelwal, A., Le, K., Mühlemann, A., Niemi, J., Shah, A., Stark, A., Wang, Y., & Wattanachit, N. (2022). Evaluation of individual and ensemble probabilistic forecasts of COVID-19 mortality in the United States. *Proceedings of the National Academy of Sciences*, *119*(15), e2113561119. https://doi.org/10.1073/pnas.2113561119

Domingos, P., & Hulten, G. (2000). Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 71–80). Association for Computing Machinery (ACM).

Gomes, H. M., Barddal, J. P., Ferreira, L. E. B., & Bifet, A. (2018). Adaptive random forests for data stream regression. In *26th European Symposium on Artificial Neural Networks, ESANN 2018, April 25–27, 2018*. i6doc.com.

Gomes, H. M., Montiel, J., Mastelini, S. M., Pfahringer, B., & Bifet, A. (2020). On ensemble techniques for data stream regression. In *2020 International Joint Conference on Neural Networks (IJCNN)* (pp. 1–8). IEEE.

Ikonomovska, E., Gama, J., & Džeroski, S. (2011). Learning model trees from evolving data streams. *Data Mining and Knowledge Discovery*, *23*(1), 128–168. https://doi.org/10.1007/s10618-010-0201-y

Ikonomovska, E., Gama, J., & Džeroski, S. (2014). Online tree-based ensembles and option trees for regression on evolving data streams. *Neurocomputing*, *150*, 458–470. https://doi.org/10.1016/j.neucom.2014.04.076

Kumar, N., & Susan, S. (2025). Non-stationary fuzzy time series modeling and forecasting using deep learning with swarm optimization. *International Journal of Machine Learning and Cybernetics*, *16*(9), 5569–5587. https://doi.org/10.1007/s13042-025-02585-1

Manapragada, C., Gomes, H. M., Salehi, M., Bifet, A., & Webb, G. I. (2022). An eager splitting strategy for online decision trees in ensembles. *Data Mining and Knowledge Discovery*, *36*(2), 566–619. https://doi.org/10.1007/s10618-021-00816-x

Manapragada, C., Webb, G. I., & Salehi, M. (2018). Extremely fast decision tree. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, August 19–23, 2018* (pp. 1953–1962). ACM.

Miralles-Pechuán, L., Kumar, A., & Suárez-Cetrulo, A. L. (2023). Forecasting COVID-19 cases using dynamic time warping and incremental machine learning methods. *Expert Systems*, *40*(6), e13237. https://doi.org/10.1111/exsy.v40.6

Moghimi, B., Kamga, C., Safikhani, A., Mudigonda, S., & Vicuna, P. (2023). Non-stationary time series model for station-based subway ridership during COVID-19 pandemic: Case study of new york city. *Transportation Research Record*, *2677*(4), 463–477. https://doi.org/10.1177/03611981221084698

Montiel, J., Bifet, A., Losing, V., Read, J., & Abdessalem, T. (2018). Learning fast and slow: A unified batch/stream framework. In *2018 IEEE International Conference on Big Data (Big Data)* (pp. 1065–1072). IEEE.

Montiel, J., Read, J., Bifet, A., & Abdessalem, T. (2018). Scikit-multiflow: A multi-output streaming framework. *Journal of Machine Learning Research*, *19*(72), 1–5.

Mouss, H., Mouss, D., Mouss, N., & Sefouhi, L. (2004). Test of page-hinckley, an approach for fault detection in an agro-alimentary production system. In *2004 5th Asian Control Conference (IEEE Cat. No. 04EX904)* (Vol. 2, pp. 815–818). IEEE.

Mydukuri, R. V., Kallam, S., Patan, R., Al-Turjman, F., & Ramachandran, M. (2022). Deming least square regressed feature selection and gaussian neuro-fuzzy multi-layered data classifier for early COVID prediction. *Expert Syst. J. Knowl. Eng.*, *39*(4), e12694. https://doi.org/10.1111/exsy.v39.4

Nguyen, D. Q., Vo, N. Q., Nguyen, T. T., Nguyen-An, K., Nguyen, Q. H., Tran, D. N., & Quan, T. T. (2022). Becaked: An explainable artificial intelligence model for COVID-19 forecasting. *Scientific Reports*, *12*(1), 7969. https://doi.org/10.1038/s41598-022-11693-9

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., &

Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Salah, I., Jouini, K., & Korbaa, O. (2023). On the use of text augmentation for stance and fake news detection. *Journal of Information and Telecommunication*, *7*(3), 359–375. https://doi.org/10.1080/24751839.2023.2198820

Tran, N. N. D., Nguyen, H. D., Huynh, N. T., Tran, N. P., & Nguyen, L. V. (2022). Segmentation on chest CT imaging in COVID-19 based on the improvement attention U-Net model. In *New trends in intelligent software methodologies, tools and techniques* (Vol. 355 of *Frontiers in Artificial Intelligence and Applications*, pp. 596–606). IOS Press.

Tran-Nguyen, M.-T., Bui, L.-D., & Do, T.-N. (2020). Decision trees using local support vector regression models for large datasets. *Journal of Information and Telecommunication*, *4*(1), 17–35. https://doi.org/10.1080/24751839.2019.1686682